EXPERIMENT NO. 3

Name: Ayush Rahul Kulkarni

Roll no: B26ET2024 Div: SY3 Class: C3

Input Code:

```
#include <stdio.h>
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}
void bubbleSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
void selectionSort(int arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min_idx = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        int temp = arr[min_idx];
        arr[min_idx] = arr[i];
```

```
        arr[i] = temp;
    }
}
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = (low - 1);
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            int temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }
    int temp = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = temp;
    return (i + 1);
}
```

```
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
void merge(int arr[], int l, int m, int r) {
    int i, j, k;
    int n1 = m - l + 1;
    int n2 = r - m;
    int L[n1], R[n2];
    for (i = 0; i < n1; i++)
        L[i] = arr[l + i];
    for (j = 0; j < n2; j++)
        R[j] = arr[m + 1 + j];
    i = 0;
    j = 0;
    k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        } else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }
```

```
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
void mergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}
int main() {
    int original[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(original) / sizeof(original[0]);
    int arr1[7], arr2[7], arr3[7], arr4[7], arr5[7];
    for(int i = 0; i < n; i++) {
        arr1[i] = original[i];
        arr2[i] = original[i];
        arr3[i] = original[i];
        arr4[i] = original[i];
        arr5[i] = original[i];
    }
```

```
    printf("Original Array: \n");
    printArray(original, n);
    bubbleSort(arr1, n);
    printf("\nSorted using Bubble Sort: \n");
    printArray(arr1, n);
    selectionSort(arr2, n);
    printf("\nSorted using Selection Sort: \n");
    printArray(arr2, n);
    insertionSort(arr3, n);
    printf("\nSorted using Insertion Sort: \n");
    printArray(arr3, n);
    quickSort(arr4, 0, n - 1);
    printf("\nSorted using Quick Sort: \n");
    printArray(arr4, n);
    mergeSort(arr5, 0, n - 1);
    printf("\nSorted using Merge Sort: \n");
    printArray(arr5, n);
    return 0;
}
```

Output:

```
Original Array:
64 34 25 12 22 11 90

Sorted using Bubble Sort:
11 12 22 25 34 64 90

Sorted using Selection Sort:
11 12 22 25 34 64 90

Sorted using Insertion Sort:
11 12 22 25 34 64 90

Sorted using Quick Sort:
11 12 22 25 34 64 90

Sorted using Merge Sort:
11 12 22 25 34 64 90
```